WEEK 13 - EXAM - MAY 4TH 2026

- **This exam is AI and Internet free**
  - No CoPilot - no Inline Completion / Suggestions
  - No online AI (ChatGPT, Gemini etc...)
  - No Browser
  - No phone
- **The Goal**: Validate you can modify, without AI, your code, which you may have written and studied with AI.

WEEK 13 - EXAM - MAY 4TH 2026 - DETAILS

- **Two Parts:**
  - **Part I**: Extending your current Moving Squares application
  - **Part II**: Implementing the **Boids** algorithm in the provided `boids_exam.py` file.
  - **Dependencies**: Exercises that depend on previous exercises are marked as such.
  - Do not feel overwhelmed by the number of exercises, some are really easy. You do not need to complete all of them to ace this exam.
- **For each exercise:**
  - You may have to write part of your answers in a new file **MY_EXAM.md** in which you will add **'## Exercise XY'** sections as needed.
  - When you write code:
    - Use Type Hints
    - Write Comments
  - **Git Ritual**: commit and push to GitHub to submit each exercise using the provided commit message.
  - If you run into issues implementing a feature, explain your thinking and analysis in **MY_EXAM.md**.
- **Do not modify your GitHub repos after the exam.**
- **Grading**:
  - Each exercise is worth between 2 and 5 points. Expect spending between 5 and 10 minutes for 2 points exercises, and between 10 to 20 minutes for 5 points exercises.
  - They add up to a total of 50 points, so the grading scale will be adjusted after correction based on the overall difficulty of the exam and the class's performance.

SETUP - GETTING EXAM-MAY-4-2026.PDF

Pull the latest from `lab1-hello-world` to get access to the **`EXAM-May-4-2026.pdf`** document (which is this presentation).

```
cd /path/to/your/lab1-hello-world
git pull

# You should get two new files:
# EXAM-May-4-2026.pdf
# boids_exam.py which will be used for Part II of the exam
```

Open `**EXAM-May-4-2026.pdf.**`

SQUARES - CLASSES VS DICTS

The exercises are using generic square objects, which in your code will either be instances of a Square class, or a dictionary.

SOME HELPERS - MATHS - RECT

- The norm of a vector V(x,y):
  - speed: float = math.hypot(x, y)
- The angle between a vector V(x,y) and the positive x axis:
  - angle: float = math.atan2(y, x) # BE CAREFUL TO THE ORDER: (y,x)
- Pygame Rectangle Collision Detection:
  -
```
# Given 2 pygame.Rect objects, rect1 & rect2
rect1 : pygame.Rect
rect2 : pygame.Rect
# rect1.collideRect(rect2) returns true
# if rect1 and rect2 are colliding
collision : bool = rect1.collideRect(rect2)
```
- **Drawing a line with Pygame:**
  -
```
surface: pygame.Surface
start_pos: Tuple[int,int] = (100,100)
end_pos: Tuple[int,int] = (200,200)
color: Type[int,int,int] = (100,100,255)
width: int = 2

pygame.draw.line(surface, color, start_pos, end_pos, w
```

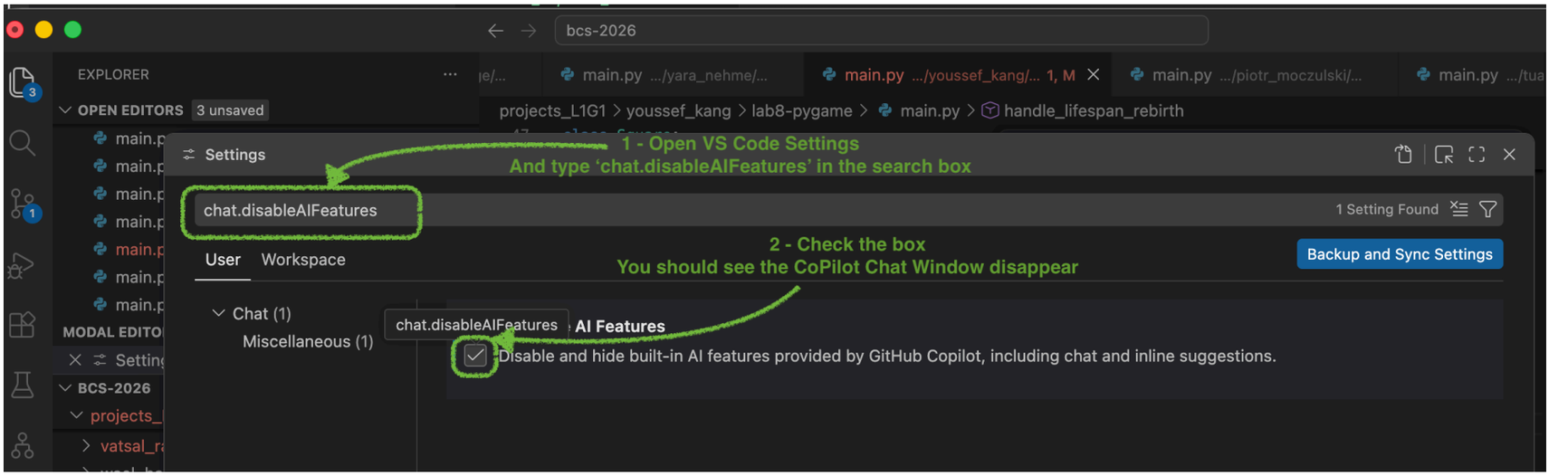
SOME HELPERS - LISTS

```
list.append(_value_, _/_)
# Add an item to the end of the list. Similar to `a[len(a):] = [x]`.

list.pop(_index=-1_, _/_)
# Remove the item at the given position in the list, and return it.
# If no index is specified, `a.pop()` removes
# and returns the last item in the list.
# It raises an [`IndexError`] if the list is empty
# or the index is outside the list range.

list.clear()
# Remove all items from the list. Similar to `del a[:]`.

list.copy()
# Return a shallow copy of the list. Similar to `a[:]`.
```

SETUP: DISABLE COPILOT

For those who have not done it yet here is a reminder on how to disable CoPilot in VS Code.

1. Open VS Code Settings
2. In the search box for Settings, type ‘chat.disableAIFeatures’
    1. Check the ‘Disable AI Features’ box. You should see the CoPilot Chat window disappear.



SETUP: INITIAL COMMIT

- **Create the MY_EXAM.md file**
  - Go to your 'lab8-pygame' folder and create an empty '**MY_EXAM.md**' file.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit -m "exam: init - MY_EXAM.md"`
  - `git push`

EXERCISE 1: A MIX OF SQUARES - 2 PTS

- **Mix Squares Feature**: Make the application start with:
  - 5 squares of size 25 pixels
  - 10 squares of size 10 pixels
  - 30 squares of size 4 pixels
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q1 – mix of squares"`
  - `git push`

EXERCISE 2: SAME SIZE RESPAWN - 2 PTS

- **Same Size Respawn Feature**: When square die, they are re-spawn **with the same size.**
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit -m "exam: q2 - same size respawn"`
  - `git push`

EXERCISE 3: SCREEN WRAPPING - 2 PTS

- **Screen Wrapping Feature**: The squares no longer bounce off the walls. Instead they wrap around the borders.
- **Specs:** - When a square 'hits' or 'passes through' a wall, it ends up on the other side of the opposite wall, speed unchanged.
- **MY_EXAM**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q3 – screen wrapping"`
  - `git push`

EXERCISE 4: COLLISION DETECTION - 2 PTS

- **Collision Detection Feature**: Implement a 'check_collision' function.
- **Specs:**
  - ```
    def check_collision(a: Square, b:Square) -> bool:
        # returns true if a and b collide
    ```
    - If you are using rectangles (most of you do), you may use the **rect.collideRect(otherRect)** method of the pygame.rect object. See 'Some Helpers' section above.
    - You may have to adapt the function signature to fit within your code. For example, if you have Square class, your check_collision function could look like this:
    - ```
      def _check_collision(self, other:Square) -> bool:
          # returns true if self and other collide
      ```
    - If you are using circles, you need to come up with your own logic: if the distance between the two centers of the two circles is less than the sum of each radius then there is a collision.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit -m "exam: q4 - collision detection"`
  - `git push`

EXERCISE 5: EATING - 3 PTS

Depends on Exercise 4.

- **Eating Feature**: Bigger squares will eat smaller squares when they are colliding.
- **Specs:**
  - Use the 'check_collision' function to detect if a bigger square can eat a smaller one.
  - Eaten squares die and get respawn with their **original size**.
- **MY_EXAM**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q5 – eating"`
  - `git push`

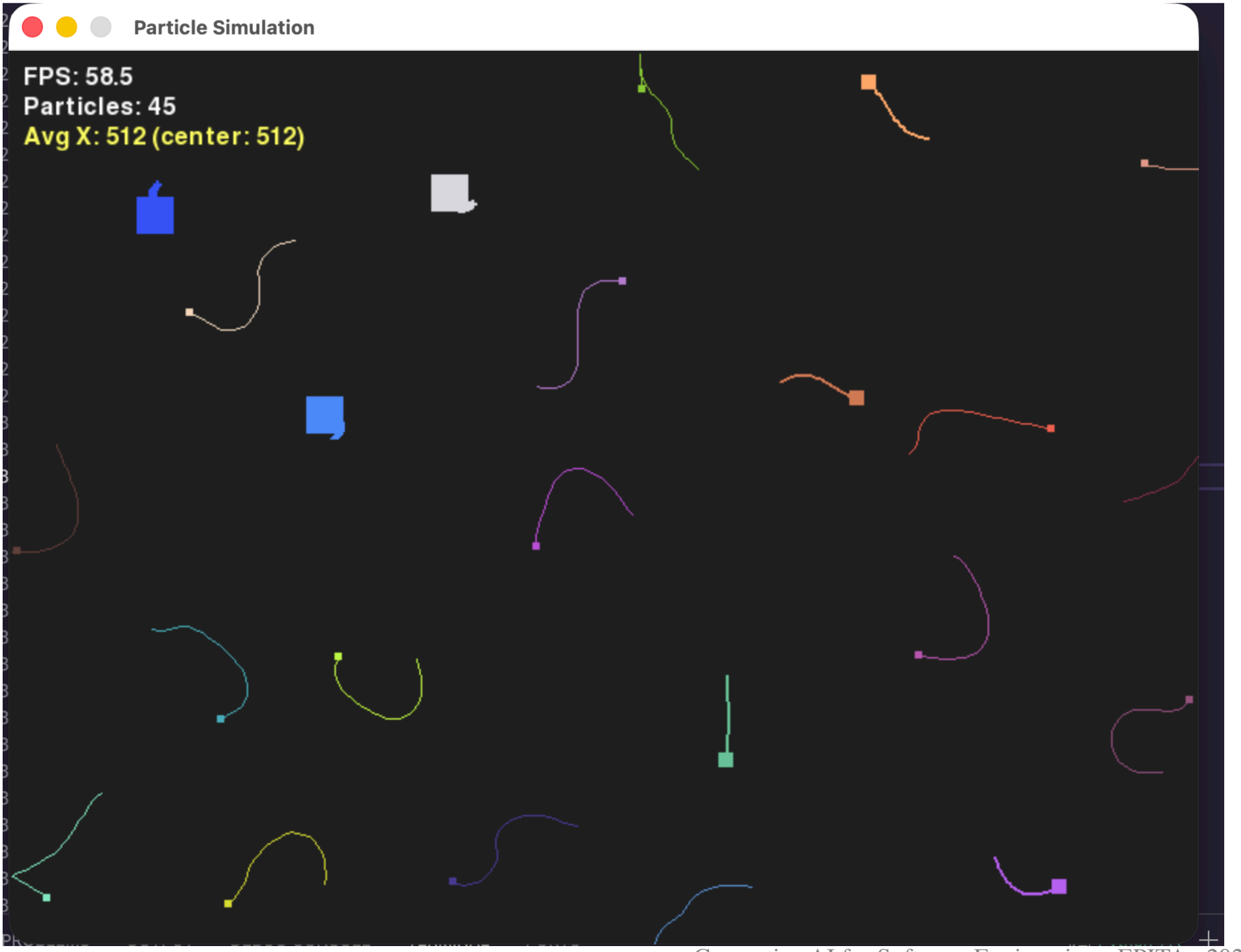
EXERCISE 6: EATING++ - 3 PTS

Depends on Exercise 5 (and therefore 4).

- **Eating++ Feature**: After eating, the predator square gets bigger by an amount proportional to the size of the prey square it just ate.
- **Specs:**
  - You decide on what 'proportional' means.
  - You may want to control the growth, that is how big a square can get.
  - Think about the speed implication for a square as it grows bigger.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q6 – eating++"`
  - `git push`

EXERCISE 7: TRAILS - 4 PTS

- **Trails Feature**: Squares have trails that are traces of their past trajectory.



EXERCISE 7: TRAILS SPECS

- **Specs:**
  - Use a constant **TRAILS_LENGTH** = 30 (but you may want to adjust it to make sure the trails are visible)
  - Each square has its trail drawn as lines joining its last **TRAILS_LENGTH** positions.
  - Use the pygame.draw.line() function - see the 'Some Helpers' section above.
- **Warning**
  - Once you have implemented the trails, you might notice some visual artifacts.
  - **TODO:**
    - Describe what is going on in MY_EXAM.
    - Explain how it could be fixed.
    - Fix the issue.
- **MY_EXAM.md**
  - Write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q7 – trails"`
  - `git push`

EXERCISE 8: SPEED TEST - 4 PTS

Warning: this exercise might be a bit difficult. It is left open on purpose.

Even if you can not implement it, write up your thinking and ideas in MY_EXAM.md.

- **Testing Speed Feature**: Implement a test that validates that a square actually moves at the the correct speed.
- **Specs:**
  - Use at least one global variable to control the test. For example: TEST_MODE_ON : bool = True
- **MY_EXAM.md**
  - Write your analysis: your assumptions, what you would evaluate exactly, and what it would take to implement it.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q8 – speed test"`
  - `git push`

EXERCISE 9: SPECIAL EFFECT - ANIMATED GROWTH - 4 PTS

Depends on Exercises 4 and 5.

- **Animated Growth Feature**: After eating, the predator square does not get bigger instantaneously, but instead, over a predefined GROWTH_SPEED constant set at 500 ms.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit -m "exam: q9 - animated growth"`
  - `git push`

PART II: THE BOIDS ALGORITHM

The **Boids algorithm** is a simple but powerful model for simulating flocking behavior—like how birds, fish, or crowds move together. It was created by Craig Reynolds in 1986.

Instead of controlling a group as a whole, the algorithm gives each individual “boid” (short for “bird-oid object”) a few local rules:

- **Separation** – avoid crowding nearby neighbors
- **Alignment** – steer toward the average direction of nearby boids
- **Cohesion** – move toward the average position of nearby boids

Even though each boid only follows these simple rules, complex and realistic group motion emerges automatically—this is a classic example of **emergent behavior**.

You are going to implement the Boids algorithm step by step.

All exercises are independent of each other.

There are plenty of hints in the code, take the time to read them.

BOIDS - SETUP

Copy the `boids_exam.py` file from `lab1-hello-world` over to your `lab8-pygame` folder.

```
cp /path/to/your/lab1-hello-world/boids_exam.py /path/to/your/lab8-pygame
```

EXERCISE 10: SCREEN WRAPPING - 2 PTS

- **Screen Wrapping Feature**: This is the same feature as what you did for the moving squares: the squares no longer bounce off the walls. Instead they wrap around the borders.
- **Specs:** - When a square 'hits' or 'passes through' a wall, it ends up on the other side of the opposite wall, speed unchanged.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q10 – boids: screen wrapping"`
  - `git push`

EXERCISE 11: RANDOM STEER - 2 PTS

- **Random Steer Feature**: Implement Random Steering of the velocity vector to create more natural movement.
- **Specs:** - Implement the 'def _random_steer(self, spread: float = 0.2) -> None:' function in boids_exam.py - Preference for angle-based steering as opposed to direct vx,vy jittering.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q11 – boids: random steer"`
  - `git push`

EXERCISE 12: SEPARATION - 5 PTS

- **Separation Feature**: Each boid steers away from nearby boids to avoid crowding.
- **Specs:**
  - Implement the `def _separation(self, boids: List['Boid']) -> pygame.Vector2:` function in `boids_exam.py`
  - Apply Separation in the `def update(self, boids: List['Boid'], dt: int) -> None:` function using a `SEPARATION_STEER_STRENGTH` coefficient.
  - Find more explanations in the source file.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit -m "exam: q12 - boids: separation"`
  - `git push`

EXERCISE 13: ALIGNMENT - 5 PTS

- **Alignment Feature**: Each boid steers toward the average direction of nearby boids.
- **Specs:**
  - Implement the `def _alignment(self, boids: List['Boid']) -> pygame.Vector2:` function in `boids_exam.py`
  - Apply Alignment in the `def update(self, boids: List['Boid'], dt: int) -> None:` function using a `ALIGNMENT_STEER_STRENGTH` coefficient.
  - Find more explanations in the source file.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit -m "exam: q13 - boids: alignment"`
  - `git push`

EXERCISE 14: COHESION - 5 PTS

- **Cohesion Feature**: Each boid steers toward the average position of nearby boids:.
- **Specs:**
  - Implement the `def _cohesion(self, boids: List['Boid']) -> pygame.Vector2:`
  - Apply Cohesion in the `def update(self, boids: List['Boid'], dt: int) -> None:` function using a `COHESION_STEER_STRENGTH` coefficient.
  - Find more explanations in the source file.
- **MY_EXAM.md**
  - No entry required but you may write up your thinking or analysis or the issues you are running into.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit -m "exam: q14 - boids: cohesion"`
  - `git push`

EXERCISE 15: S.A.C. TEST - ANALYSIS - 2 PTS

S.A.C. = Separation - Alignment - Cohesion.

What is expected is an analysis in MY_EXAM.md.

- **S.A.C Test Feature**: A test that validates that boid particles are indeed flocking.
- **MY_EXAM.md**
  - Write your analysis: what your would be measuring and testing exactly and how you would go about it.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q15 – boids: s.a.c. test analysis"`
  - `git push`

EXERCISE 16: S.A.C. TEST - IMPLEMENTATION - 3 PTS

S.A.C. = Separation - Alignment - Cohesion.

- **S.A.C Test Feature**: A test that validates that boid particles are indeed flocking.
- **Specs:**:
  - Implement the S.A.C. test that you have described in Exercise 15.
- **MY_EXAM.md**
  - Write your analysis: what your would be measuring and testing exactly and how you would go about it.
- **Git/GitHub Ritual**
  - `git add .`
  - `git commit –m "exam: q16 – boids: s.a.c. test implementation"`
  - `git push`

TIME'S ALMOST UP

CLEAN UP AND SUBMIT

- Make sure that your applications (main.py and boids_exam.py) are working, even if you have not finished a particular exercise.
- You may comment out the part that break the app. Add comments to explain.
- You may explain more in **MY_EXAM.md**.
- **Final Git Ritual**
  - `git add .`
  - `git commit -m "exam: final submission"`
  - `git push`
- Send me the link to your repo by email **now**.
- Do not touch your repos.